

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 04

NOMBRE COMPLETO: Pablo Sebastian Noyola Torres

Nº de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027 - 1

FECHA DE ENTREGA LÍMITE: 19 de septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Terminar el Rover con:

-Cabina

-Base

-Brazo con pinza con teclado se puede girar cada articulación

-6 llantas con teclado se pueden girar cada una de las 6 llantas por separado

Capturas de código:

```
int main()
{
    mainWindow = Window(800, 600);
    mainWindow.Initialise();
    //Cilindro y cono reciben resolución (slices, rebanadas) y Radio de circunferencia de la base y tapa

    CrearCubo();//índice 0 en MeshList
    CrearPiramideTriangular();//índice 1 en MeshList
    CrearCilindro(18, 1.0f);//índice 2 en MeshList
    CrearCono(25, 2.0f);//índice 3 en MeshList
    CrearPiramideCuadrangular();//índice 4 en MeshList
    CreateShaders();
```

```
//articulacion1 hasta articulación5 sólo son puntos de rotación o articulación, en este caso no dibujaremos esferas q
model = glm::mat4(1.0);

model = glm::translate(model, glm::vec3(0.0f, 4.0f, -4.0f)); //NOS POSICIONAMOS EN EL CENTRO DEL OBJETO X AMARILLA
modelaux = model; //guardamos la matriz de modelo para que la base se mueva con el objeto
// Creando la cabina del rover ¿CUÁNTAS UNIDADES MEDIRÁ EN Z?
model = glm::translate(model, glm::vec3(1.0f, 2.0f, 0.0f));//PARA LLEGAR AL CENTRO DE LA CABINA a PARTIR DEL ORIGEN
modelaux = model;
model = glm::scale(model, glm::vec3(8.0f, 4.0f, 6.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución se programe cambio entre proyección
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.84f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
//meshList[3]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera


// BASE
//model = glm::mat4(1.0); //Si dejamos está línea, la base no estará unida al centro del objeto. se debe comentar par
/*En su lugar usamos la matriz auxiliar modelaux para que la base se mueva con el objeto.
Lo que debemos de saber es: de las transformaciones geométricas que se aplican a la cabina y
cuales queremos que se apliquen a la base. En este caso, sólo queremos que se aplique la traslación del origen, no la
Por lo tanto, debemos de guardar en modelaux sólo la traslación del origen y luego aplicarla a la base.
*/
model = modelaux;
//NOS POSICIONAMOS EN EL CENTRO DE LA BASE
model = glm::translate(model, glm::vec3(-1.0f, -2.75f, 0.0f));
modelaux = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
model = glm::scale(model, glm::vec3(10.0f, 1.5f, 8.0f));
```

```
385         modelaux = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
386         model = glm::scale(model, glm::vec3(10.0f, 1.5f, 8.0f));
387         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
388         color = glm::vec3(0.61f, 0.63f, 0.65f);
389         glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
390         meshList[0]->RenderMesh();
391
392         model = modelaux;
393         modelaux2 = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
394         /*
395         A PARTIR DE ESTE PUNTO HAY QUE ACOMODAR VARIOS ELEMENTOS:
396         LA BASE PARA EL BRAZO
397         LA BASE PARA LA LLANTA FRONTAL IZQUIERDA/ CERCA
398         LA BASE PARA LA LLANTA DE EN MEDIO IZQUIERDA/ CERCA
399         LA BASE PARA LA LLANTA DE ATRÁS IZQUIERDA / CERCA
400         .........¿FALTA ALGO MÁS? sí, X2 todas las llantas
401         LA BASE PARA LA LLANTA FRONTAL DERECHA /LEJOS
402         LA BASE PARA LA LLANTA DE EN MEDIO DERECHA /LEJOS
403         LA BASE PARA LA LLANTA DE ATRÁS DERECHA  /LEJOS
404
405         CÓMO ESTAREMOS REGRESANDO A ESTE PUNTO EN ESPECÍFICO VARIAS VECES, DEBEMOS DE USAR UNA NUEVA MATRIZ AUXILIAR PARA CADA UNO DE LOS ELEMENTOS
406         QUE SE QUIERAN CONECTAR A LA BASE, YA QUE SI USAMOS LA MISMA MATRIZ AUXILIAR, SE SOBRESCRIBIRÁN LAS TRANSFORMACIONES GEOMÉTRICAS DE CADA ELEMENTO
407         Y NO SE PODRÁN CONECTAR CORRECTAMENTE.
408
409         */
410
411
412         // Brazo Principal------------------------------------------------------------------------------------------------------------------------
413
414         //BASE PARA EL BRAZO
415         model = modelaux2;
416         model = glm::translate(model, glm::vec3(-3.5f, 1.25f, 2.5f));
417         //modelaux = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
418         //model = glm::scale(model, glm::vec3(10.0f, 1.5f, 8.0f)); NO se requirio escalar el cubo
419         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
```

```
419         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
420         color = glm::vec3(0.32f, 0.33f, 0.35f);
421         glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
422         meshList[0]->RenderMesh();
423
424         //EJE DE ROTACIÓN
425         model = glm::translate(model, glm::vec3(0.0f, 0.75f, 0.0f));
426         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
427         modelaux = model;
428         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
429         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
430         color = glm::vec3(0.80f, 0.82f, 0.84f);
431         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
432         sp.render();
433
434         //Brazo
435         model = modelaux;
436         model = glm::translate(model, glm::vec3(-1.25f, 1.25f, 0.0f));
437         model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
438         modelaux = model;
439         model = glm::scale(model, glm::vec3(0.5f, 3.0f, 0.5f));
440         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
441         color = glm::vec3(1.0f, 0.84f, 0.0f);
442         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
443         meshList[2]->RenderMeshGeometry();
444
445         //eje br1-brz2
446         model = modelaux;
447         model = glm::translate(model, glm::vec3(0.0f, 1.5f, 0.0f));
448         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
449         modelaux = model;
450         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
451         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
452         color = glm::vec3(0.80f, 0.82f, 0.84f);
453         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        sp.render();

        // Brazo 2
        model = modelaux;
        model = glm::translate(model, glm::vec3(-1.25f, 1.25f, 0.0f));
        model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(0.5f, 3.0f, 0.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(1.0f, 0.84f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[2]->RenderMeshGeometry();

        //eje br1-brz2
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.0f, 1.5f, 0.0f));
        model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 1.0f, 0.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(1.2f, 0.4f, 1.2f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.2f, 0.2f, 0.2f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMesh();
        ///terminar

        //eje br1-brz2
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.0f, .5f, 0.0f));
        model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.09f, 0.10f, 0.12f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        sp.render();

        // Mano de pinzas
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.5f, 0.5f, 0.0f));
        model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.61f, 0.63f, 0.65f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMeshGeometry();

        // Dedo 1
        model = modelaux;
        model = glm::translate(model, glm::vec3(-0.8f, 1.2f, 0.0f));
        model = glm::rotate(model, glm::radians(25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
        model = glm::scale(model, glm::vec3(0.4f, 1.4f, 0.4f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.2f, 0.2f, 0.2f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMeshGeometry();

        // Dedo 2
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.8f, 1.2f, 0.0f));
        model = glm::rotate(model, glm::radians(-25.0f), glm::vec3(0.0f, 0.0f, 1.0f));
        model = glm::scale(model, glm::vec3(0.4f, 1.4f, 0.4f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.2f, 0.2f, 0.2f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMeshGeometry();

        // Dedo 3
```

```
        // Dedo 3
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.0f, 1.2f, -0.8f));
        model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));
        model = glm::scale(model, glm::vec3(0.4f, 1.4f, 0.4f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.2f, 0.2f, 0.2f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMeshGeometry();
        // Brazo Principal----------------------------------------------------------------------------------------------------


        // Para ruedas derechas ----------------------------------------------------------------------------------------------

        //BASE PARA EL BRAZO
        model = modelaux2;
        model = glm::translate(model, glm::vec3(0.5f, 0.0f, 4.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.32f, 0.33f, 0.35f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMesh();

        //EJE DE ROTACIÓN
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.75f));
        model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.80f, 0.82f, 0.84f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        sp.render();

        // Brazo
        model = modelaux;
```

```
        model = glm::translate(model, glm::vec3(-1.0f, 0.0f, 0.0f));
        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.61f, 0.63f, 0.65f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMeshGeometry();

        // Segunda parte de brazo
        model = modelaux;
        model = glm::translate(model, glm::vec3(-.6f, 1.0f, 0.0f));
        model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.61f, 0.63f, 0.65f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMeshGeometry();

        // Eje de rueda
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.0f, -0.8f, 0.4f));
        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
        model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 1.0f, 0.0f));
        modelaux = model;
        model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.55f, 0.57f, 0.60f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[2]->RenderMeshGeometry();

        // Llanta
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
```

```
590         model = glm::scale(model, glm::vec3(0.6f, 0.25f, 0.6f));
591         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
592         color = glm::vec3(0.15f, 0.15f, 0.15f);
593         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
594         meshList[2]->RenderMeshGeometry();
595         model = modelaux;
596         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
597         model = glm::scale(model, glm::vec3(0.53f, 0.252f, 0.53f));
598         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
599         color = glm::vec3(0.90f, 0.90f, 0.88f);
600         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
601         meshList[2]->RenderMeshGeometry();
602
603
604         //-------------------------------------------------------------------------------------------------------
605         //BASE PARA EL BRAZO
606         model = modelaux2;
607         model = glm::translate(model, glm::vec3(1.5f, 0.0f, 4.5f));
608         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
609         color = glm::vec3(0.32f, 0.33f, 0.35f);
610         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
611         meshList[0]->RenderMesh();
612
613         //EJE DE ROTACIÓN
614         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.75f));
615         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
616         modelaux = model;
617         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
618         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
619         color = glm::vec3(0.80f, 0.82f, 0.84f);
620         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
621         sp.render();
622
623         //Brazo
624         model = modelaux;
```

```
625         model = glm::translate(model, glm::vec3(1.0f, 0.0f, 0.0f));
626         model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
627         modelaux = model;
628         model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
629         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
630         color = glm::vec3(0.61f, 0.63f, 0.65f);
631         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
632         meshList[0]->RenderMeshGeometry();
633
634         // Segunda parte de brazo
635         model = modelaux;
636         model = glm::translate(model, glm::vec3(-.6f, -1.0f, 0.0f));
637         model = glm::rotate(model, glm::radians(-45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
638         modelaux = model;
639         model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
640         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
641         color = glm::vec3(0.61f, 0.63f, 0.65f);
642         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
643         meshList[0]->RenderMeshGeometry();
644
645         // Eje de llanta
646         model = modelaux;
647         model = glm::translate(model, glm::vec3(0.0f, -0.8f, 0.4f));
648         model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
649         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 1.0f, 0.0f));
650         modelaux = model;
651         model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
652         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
653         color = glm::vec3(0.55f, 0.57f, 0.60f);
654         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
655         meshList[2]->RenderMeshGeometry();
656
657         // Llanta
658         model = modelaux;
659         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
```

```
659         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
660         model = glm::scale(model, glm::vec3(0.6f, 0.25f, 0.6f));
661         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
662         color = glm::vec3(0.15f, 0.15f, 0.15f);
663         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
664         meshList[2]->RenderMeshGeometry();
665         model = modelaux;
666         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
667         model = glm::scale(model, glm::vec3(0.53f, 0.252f, 0.53f));
668         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
669         color = glm::vec3(0.90f, 0.90f, 0.88f);
670         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
671         meshList[2]->RenderMeshGeometry();
672
673         // Para ruedas derechas ----------------------------------------------------------------------------------------
674
675         // Para ruedas izquierdas --------------------------------------------------------------------------------------
676
677         //BASE PARA EL BRAZO
678         model = modelaux2;
679         model = glm::translate(model, glm::vec3(0.5f, 0.0f, -4.5f));
680         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
681         color = glm::vec3(0.32f, 0.33f, 0.35f);
682         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
683         meshList[0]->RenderMesh();
684
685         //EJE DE ROTACIÓN
686         model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.75f));
687         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
688         modelaux = model;
689         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
690         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
691         color = glm::vec3(0.80f, 0.82f, 0.84f);
692         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
693         sp.render();
```

```
693         sp.render();
694
695         //Brazo
696         model = modelaux;
697         model = glm::translate(model, glm::vec3(-1.0f, 0.0f, 0.0f));
698         model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
699         modelaux = model;
700         model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
701         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
702         color = glm::vec3(0.61f, 0.63f, 0.65f);
703         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
704         meshList[0]->RenderMeshGeometry();
705
706         // Segunda parte de brazo
707         model = modelaux;
708         model = glm::translate(model, glm::vec3(-.6f, 1.0f, 0.0f));
709         model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
710         modelaux = model;
711         model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
712         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
713         color = glm::vec3(0.61f, 0.63f, 0.65f);
714         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
715         meshList[0]->RenderMeshGeometry();
716
717         // Eje de llanta
718         model = modelaux;
719         model = glm::translate(model, glm::vec3(0.0f, -0.8f, -0.4f));
720         model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
721         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 1.0f, 0.0f));
722         modelaux = model;
723         model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
724         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
725         color = glm::vec3(0.55f, 0.57f, 0.60f);
726         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
727         meshList[2]->RenderMeshGeometry();
```

```
727         meshList[2]->RenderMeshGeometry();
728
729         // Llanta
730         model = modelaux;
731         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
732         model = glm::scale(model, glm::vec3(0.6f, 0.25f, 0.6f));
733         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
734         color = glm::vec3(0.15f, 0.15f, 0.15f);
735         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
736         meshList[2]->RenderMeshGeometry();
737         model = modelaux;
738         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
739         model = glm::scale(model, glm::vec3(0.53f, 0.252f, 0.53f));
740         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
741         color = glm::vec3(0.90f, 0.90f, 0.88f);
742         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
743         meshList[2]->RenderMeshGeometry();
744
745         //-------------------------------------------------------------------------------------------------
746         //BASE PARA EL BRAZO
747         model = modelaux2;
748         model = glm::translate(model, glm::vec3(1.5f, 0.0f, -4.5f));
749         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
750         color = glm::vec3(0.32f, 0.33f, 0.35f);
751         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
752         meshList[0]->RenderMesh();
753
754         //EJE DE ROTACIÓN
755         model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.75f));
756         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, 1.0f));
757         modelaux = model;
758         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
759         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
760         color = glm::vec3(0.80f, 0.82f, 0.84f);
761         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
762         sp.render();
763
764         //Brazo
765         model = modelaux;
766         model = glm::translate(model, glm::vec3(1.0f, 0.0f, 0.0f));
767         model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
768         modelaux = model;
769         model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
770         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
771         color = glm::vec3(0.61f, 0.63f, 0.65f);
772         glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
773         meshList[0]->RenderMeshGeometry();
774
775
776         // Segunda parte del brazo
777         model = modelaux;
778         model = glm::translate(model, glm::vec3(-.6f, -1.0f, 0.0f));
779         model = glm::rotate(model, glm::radians(-45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
780         modelaux = model;
781         model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
782         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
783         color = glm::vec3(0.61f, 0.63f, 0.65f);
784         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
785         meshList[0]->RenderMeshGeometry();
786
787         // Eje de llanta
788         model = modelaux;
789         model = glm::translate(model, glm::vec3(0.0f, -0.8f, -0.4f));
790         model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
791         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()), glm::vec3(0.0f, 1.0f, 0.0f));
792         modelaux = model;
793         model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
794         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
795         color = glm::vec3(0.55f, 0.57f, 0.60f);
796         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
797         meshList[2]->RenderMeshGeometry();
798
799         // Llanta
800         model = modelaux;
801         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
802         model = glm::scale(model, glm::vec3(0.6f, 0.25f, 0.6f));
803         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
804         color = glm::vec3(0.15f, 0.15f, 0.15f);
805         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
806         meshList[2]->RenderMeshGeometry();
807         model = modelaux;
808         model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
809         model = glm::scale(model, glm::vec3(0.53f, 0.252f, 0.53f));
810         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
811         color = glm::vec3(0.90f, 0.90f, 0.88f);
812         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
813         meshList[2]->RenderMeshGeometry();
814
815
816         // Llantas delanteras ----------------------------------------------------------------------------------------------
817         // Llanta delantera derecha----------------------------------------------------------------------------------------
818         //BASE PARA EL BRAZO
819         model = modelaux2;
820         model = glm::translate(model, glm::vec3(-4.5f, 0.0f, 4.5f));
821         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
822         color = glm::vec3(0.32f, 0.33f, 0.35f);
823         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
824         meshList[0]->RenderMesh();
825
826         //EJE DE ROTACIÓN
827         model = glm::translate(model, glm::vec3(0.0f, 0.75f, 0.0f));
828         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion13()), glm::vec3(0.0f, 0.0f, 1.0f));
829         modelaux = model;
830         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
831         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
```

```
831         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
832         color = glm::vec3(0.80f, 0.82f, 0.84f);
833         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
834         sp.render();
835
836         //Brazo
837         model = modelaux;
838         model = glm::translate(model, glm::vec3(-1.25f, -0.6f, 0.0f));
839         model = glm::rotate(model, glm::radians(-50.0f), glm::vec3(0.0f, 0.0f, 1.0f));
840         modelaux = model;
841         model = glm::scale(model, glm::vec3(0.5f, 3.0f, 0.5f));
842         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
843         color = glm::vec3(1.0f, 0.84f, 0.0f);
844         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
845         meshList[0]->RenderMeshGeometry();
846
847         // EJE DE ROTACIÓN
848         model = modelaux;
849         model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
850         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion14()), glm::vec3(0.0f, 0.0f, 1.0f));
851         modelaux = model;
852         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
853         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
854         color = glm::vec3(0.80f, 0.82f, 0.84f);
855         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
856         sp.render();
857
858
859         // Brazo
860         model = modelaux;
861         model = glm::translate(model, glm::vec3(-.0f, -0.6f, 0.0f));
862         model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 0.0f, 1.0f));
863         modelaux = model;
864         model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
865         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
```

```
865             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
866             color = glm::vec3(0.61f, 0.63f, 0.65f);
867             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
868             meshList[0]->RenderMeshGeometry();
869 
870             // Parte 2 del brazo
871             model = modelaux;
872             model = glm::translate(model, glm::vec3(.6f, -.25f, 0.0f));
873             model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
874             modelaux = model;
875             model = glm::scale(model, glm::vec3(0.5f, 1.2f, 0.5f));
876             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
877             color = glm::vec3(0.61f, 0.63f, 0.65f);
878             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
879             meshList[0]->RenderMeshGeometry();
880 
881             // Eje de llanta
882             model = modelaux;
883             model = glm::translate(model, glm::vec3(0.0f, 0.4f, 0.35f));
884             model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
885             model = glm::rotate(model, glm::radians(mainWindow.getarticulacion15()), glm::vec3(0.0f, 1.0f, 0.0f));
886             modelaux = model;
887             model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
888             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
889             color = glm::vec3(0.55f, 0.57f, 0.60f);
890             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
891             meshList[2]->RenderMeshGeometry();
892 
893             // Llanta
894             model = modelaux;
895             model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
896             model = glm::scale(model, glm::vec3(0.6f, 0.25f, 0.6f));
897             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
898             color = glm::vec3(0.15f, 0.15f, 0.15f);
899             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

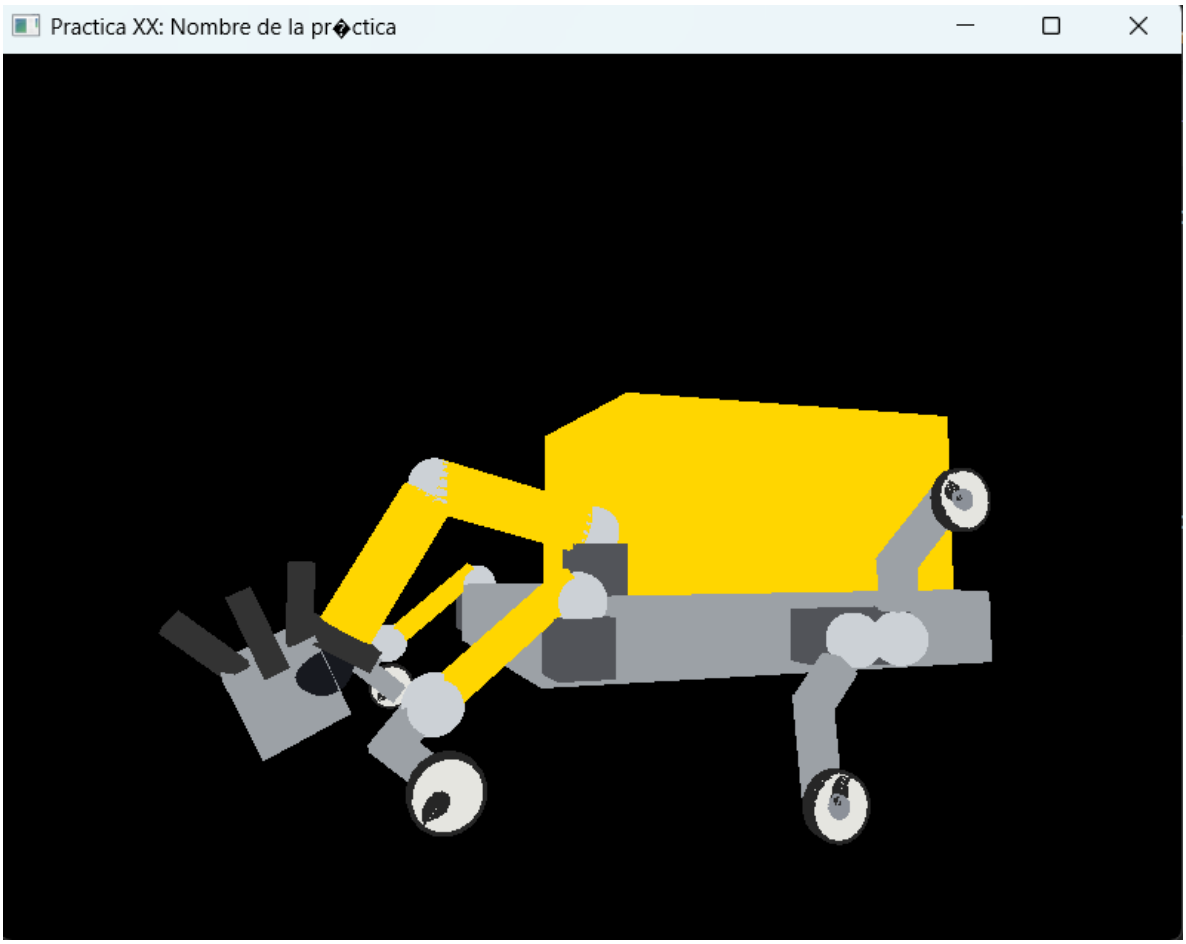
```
898             color = glm::vec3(0.15f, 0.15f, 0.15f);
899             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
900             meshList[2]->RenderMeshGeometry();
901             model = modelaux;
902             model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
903             model = glm::scale(model, glm::vec3(0.53f, 0.252f, 0.53f));
904             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
905             color = glm::vec3(0.90f, 0.90f, 0.88f);
906             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
907             meshList[2]->RenderMeshGeometry();
908 
909             // Llanta delantera derecha----------------------------------------------------------------------------------
910 
911             // Llanta delantera izquierda--------------------------------------------------------------------------------
912             //BASE PARA EL BRAZO
913             model = modelaux2;
914             model = glm::translate(model, glm::vec3(-4.5f, 0.0f, -4.5f));
915             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
916             color = glm::vec3(0.32f, 0.33f, 0.35f);
917             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
918             meshList[0]->RenderMesh();
919 
920             //EJE DE ROTACIÓN
921             model = glm::translate(model, glm::vec3(0.0f, 0.75f, 0.0f));
922             model = glm::rotate(model, glm::radians(mainWindow.getarticulacion16()), glm::vec3(0.0f, 0.0f, 1.0f));
923             modelaux = model;
924             model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
925             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
926             color = glm::vec3(0.80f, 0.82f, 0.84f);
927             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
928             sp.render();
929 
930             //Brazo
931             model = modelaux;
932             model = glm::translate(model, glm::vec3(-1.25f, -0.6f, 0.0f));
```

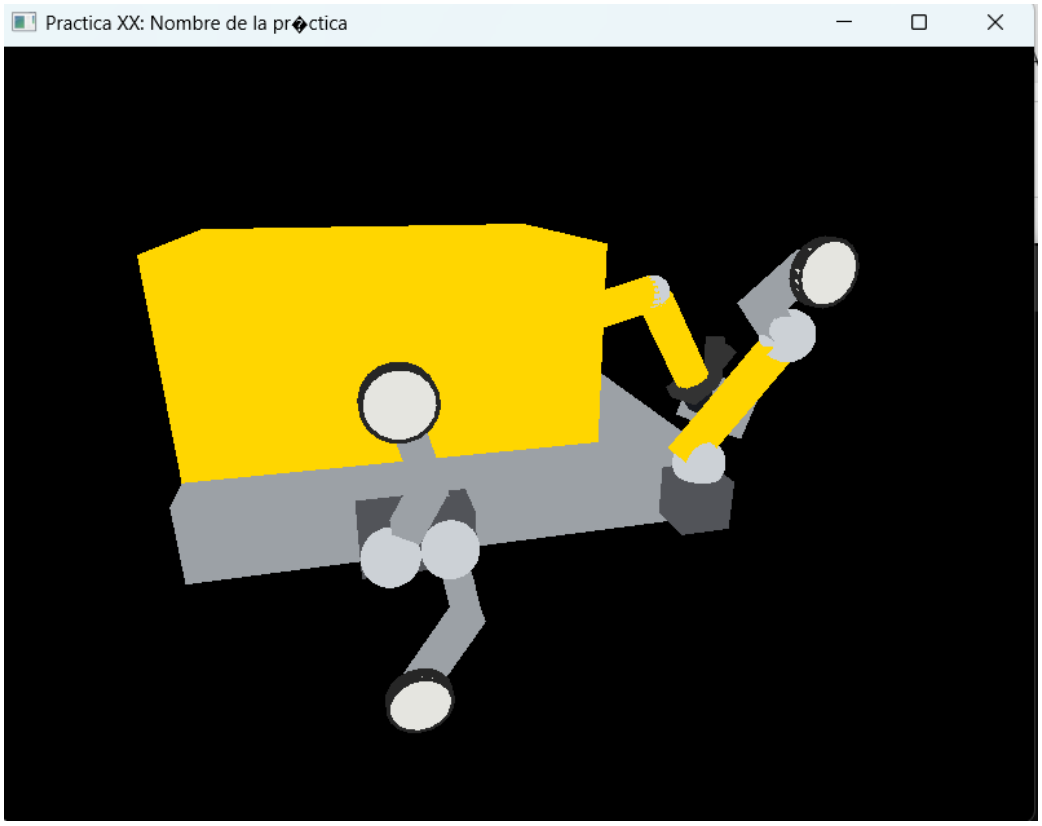
```
931         model = modelaux;
932         model = glm::translate(model, glm::vec3(-1.25f, -0.6f, 0.0f));
933         model = glm::rotate(model, glm::radians(-50.0f), glm::vec3(0.0f, 0.0f, 1.0f));
934         modelaux = model;
935         model = glm::scale(model, glm::vec3(0.5f, 3.0f, 0.5f));
936         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
937         color = glm::vec3(1.0f, 0.84f, 0.0f);
938         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
939         meshList[0]->RenderMeshGeometry();
940
941         // EJE DE ROTACIÓN
942         model = modelaux;
943         model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
944         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion17()), glm::vec3(0.0f, 0.0f, 1.0f));
945         modelaux = model;
946         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
947         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
948         color = glm::vec3(0.80f, 0.82f, 0.84f);
949         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
950         sp.render();
951
952
953         //Brazo
954         model = modelaux;
955         model = glm::translate(model, glm::vec3(-.0f, -0.6f, 0.0f));
956         model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 0.0f, 1.0f));
957         modelaux = model;
958         model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
959         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
960         color = glm::vec3(0.61f, 0.63f, 0.65f);
961         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
962         meshList[0]->RenderMeshGeometry();
963
964         // Parte 2 del brazo
965         model = modelaux;
```

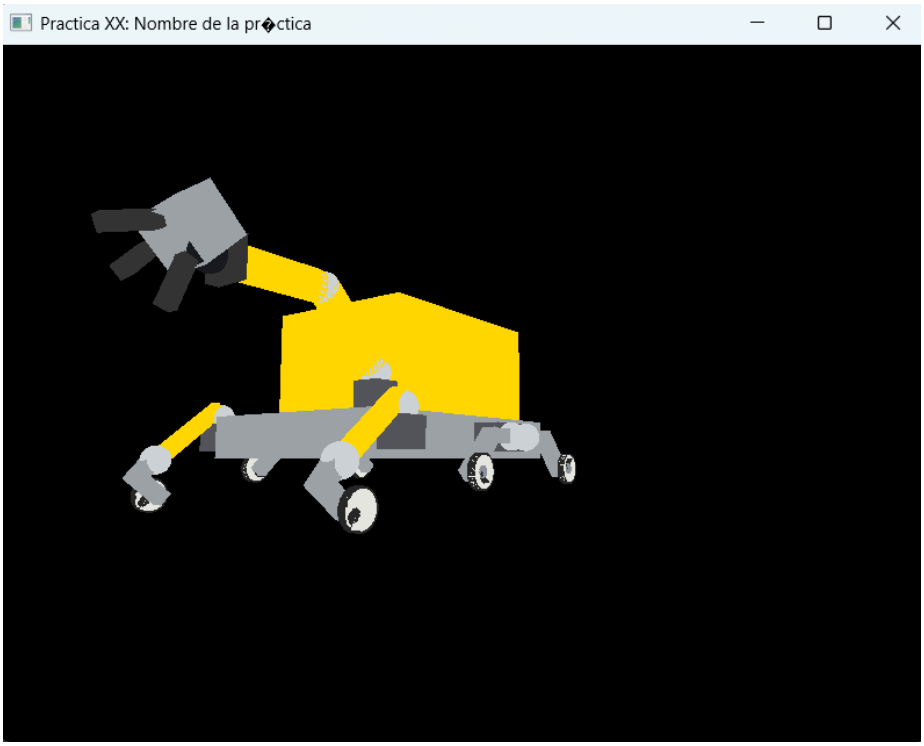
```
965         model = modelaux;
966         model = glm::translate(model, glm::vec3(.6f, -.25f, 0.0f));
967         model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
968         modelaux = model;
969         model = glm::scale(model, glm::vec3(0.5f, 1.2f, 0.5f));
970         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
971         color = glm::vec3(0.61f, 0.63f, 0.65f);
972         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
973         meshList[0]->RenderMeshGeometry();
974
975         // Eje de llanta
976         model = modelaux;
977         model = glm::translate(model, glm::vec3(0.0f, 0.4f, -0.35f));
978         model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
979         model = glm::rotate(model, glm::radians(mainWindow.getarticulacion18()), glm::vec3(0.0f, 1.0f, 0.0f));
980         modelaux = model;
981         model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
982         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
983         color = glm::vec3(0.55f, 0.57f, 0.60f);
984         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
985         meshList[2]->RenderMeshGeometry();
986
987         // Llanta
988         model = modelaux;
989         model = glm::translate(model, glm::vec3(0.0f, -0.2f, 0.0f));
990         model = glm::scale(model, glm::vec3(0.6f, 0.25f, 0.6f));
991         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
992         color = glm::vec3(0.15f, 0.15f, 0.15f);
993         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
994         meshList[2]->RenderMeshGeometry();
995         model = modelaux;
996         model = glm::translate(model, glm::vec3(0.0f, -0.2f, 0.0f));
997         model = glm::scale(model, glm::vec3(0.53f, 0.252f, 0.53f));
998         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
999         color = glm::vec3(0.90f, 0.90f, 0.88f);
```

```
987             // Llanta
988             model = modelaux;
989             model = glm::translate(model, glm::vec3(0.0f, -0.2f, 0.0f));
990             model = glm::scale(model, glm::vec3(0.6f, 0.25f, 0.6f));
991             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
992             color = glm::vec3(0.15f, 0.15f, 0.15f);
993             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
994             meshList[2]->RenderMeshGeometry();
995             model = modelaux;
996             model = glm::translate(model, glm::vec3(0.0f, -0.2f, 0.0f));
997             model = glm::scale(model, glm::vec3(0.53f, 0.252f, 0.53f));
998             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
999             color = glm::vec3(0.90f, 0.90f, 0.88f);
1000            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1001            meshList[2]->RenderMeshGeometry();
1002            // Llanta delantera izquierda----------------------------------------
1003
1004            glUseProgram(0);
1005            mainWindow.swapBuffers();
1006        }
1007        return 0;
1008    }
```

Capturas de ejecución:







Video de ejecución: youtube.com/watch?v=h5QXDJIlF_w&sttick=0

En este ejercicio se desarrolló y completó con éxito el modelo tridimensional de un Rover utilizando OpenGL y GLM, cumpliendo detalladamente con todos los requerimientos establecidos para la práctica. La estructura jerárquica del robot integra una cabina principal y una base sólida interconectadas mediante el uso estratégico de matrices auxiliares (modelaux y modelaux2), lo que permite mantener la coherencia espacial y evitar la sobreescritura de transformaciones geométricas. Asimismo, se incorporó un complejo brazo robótico equipado con pinzas y múltiples dedos, cuyas articulaciones responden de manera fluida y dinámica a las entradas del teclado gracias a métodos de control independientes que permiten girar cada articulación por separado mediante funciones de ángulo como getarticulacion1() hasta getarticulacion18(). Finalmente, el diseño contempla un sistema completo de 6 llantas, donde cada una cuenta con sus respectivos ejes de rotación, soportes laterales y rines, logrando que cada una de las 6 llantas pueda girar y controlarse por separado de forma individual desde el teclado, consolidando así un modelo interactivo, funcional y preciso dentro del entorno gráfico.

2.- Crear Jerárquicamente una sonda espacial

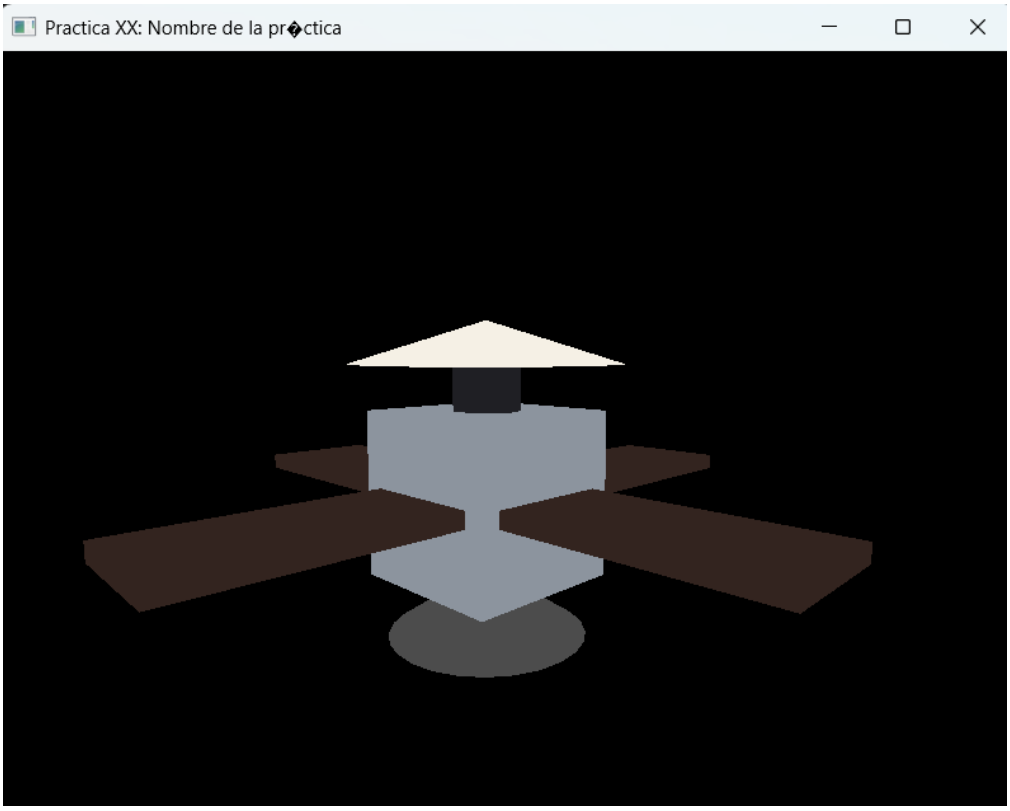
Capturas de código:

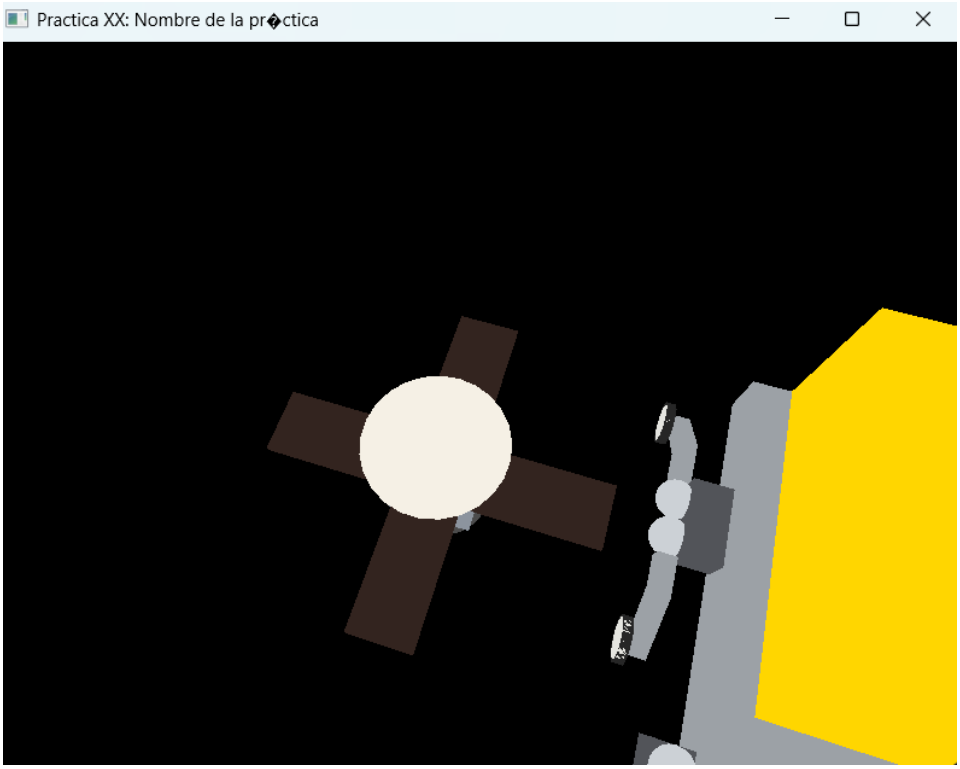
```
1006            // Sonda espacial----------------------------------------------------------------------------------------
1007
1008            // Cuerpo principal
1009            model = glm::mat4(1.0f);
1010            model = glm::translate(model, glm::vec3(0.0f, 5.0f, -15.0f));
1011            model = glm::rotate(model, glm::radians(mainWindow.getarticulacion19()), glm::vec3(1.0f, 1.0f, 1.0f));
1012            modelaux = model;
1013            model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
1014            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1015            color = glm::vec3(0.55f, 0.58f, 0.62f);
1016            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1017            meshList[0]->RenderMesh();
1018
1019            // 4 PANELES SOLARES
1020
1021            // Panel izquierdo
1022            model = modelaux;
1023            model = glm::translate(model, glm::vec3(-2.5f, 0.0f, 0.0f));
1024            model = glm::rotate(model, glm::radians(mainWindow.getarticulacion20()), glm::vec3(1.0f, 0.0f, 0.0f));
1025            model = glm::scale(model, glm::vec3(3.0f, 0.2f, 1.5f));
1026            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1027            color = glm::vec3(0.20f, 0.14f, 0.12f);
1028            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1029            meshList[0]->RenderMesh();
1030
1031
1032            // Panel derecho
1033            model = modelaux;
1034            model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
1035            model = glm::rotate(model, glm::radians(mainWindow.getarticulacion21()), glm::vec3(1.0f, 0.0f, 0.0f));
1036            model = glm::scale(model, glm::vec3(3.0f, 0.2f, 1.5f));
1037            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1038            color = glm::vec3(0.20f, 0.14f, 0.12f);
1039            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1040            meshList[0]->RenderMesh();
```

```
1040            meshList[0]->RenderMesh();
1041
1042
1043            // Panel frontal
1044            model = modelaux;
1045            model = glm::translate(model, glm::vec3(0.0f, 0.0f, 2.5f));
1046            model = glm::rotate(model, glm::radians(mainWindow.getarticulacion22()), glm::vec3(0.0f, 0.0f, 1.0f));
1047            model = glm::scale(model, glm::vec3(1.5f, 0.2f, 3.0f));
1048            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1049            color = glm::vec3(0.20f, 0.14f, 0.12f);
1050            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1051            meshList[0]->RenderMesh();
1052
1053
1054            // Panel trasero
1055            model = modelaux;
1056            model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.5f));
1057            model = glm::rotate(model, glm::radians(mainWindow.getarticulacion23()), glm::vec3(0.0f, 0.0f, 1.0f));
1058            model = glm::scale(model, glm::vec3(1.5f, 0.2f, 3.0f));
1059            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1060            color = glm::vec3(0.20f, 0.14f, 0.12f);
1061            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1062            meshList[0]->RenderMesh();
1063
1064            // Para antena superior
1065
1066            // Cilindro
1067            model = modelaux;
1068            model = glm::translate(model, glm::vec3(0.0f, 1.3f, 0.0f));
1069            modelaux2 = model;
1070            model = glm::scale(model, glm::vec3(0.4f, 0.8f, 0.4f));
1071            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1072            color = glm::vec3(0.12f, 0.12f, 0.14f);
1073            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1074            meshList[2]->RenderMeshGeometry();
```

```
1074            meshList[2]->RenderMeshGeometry();
1075
1076            // Antena
1077            model = modelaux2;
1078            model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
1079            model = glm::rotate(model, glm::radians(mainWindow.getarticulacion24()), glm::vec3(0.0f, 1.0f, 0.0f));
1080            model = glm::scale(model, glm::vec3(0.8f, 0.5f, 0.8f));
1081            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1082            color = glm::vec3(0.96f, 0.94f, 0.90f);
1083            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1084            meshList[3]->RenderMeshGeometry();
1085
1086            // Propulsor
1087            model = modelaux;
1088            model = glm::translate(model, glm::vec3(0.0f, -1.3f, 0.0f));
1089            model = glm::scale(model, glm::vec3(0.6f, 0.8f, 0.6f));
1090            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1091            color = glm::vec3(0.3f, 0.3f, 0.3f);
1092            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1093            meshList[3]->RenderMeshGeometry();
1094
1095            // Sonda espacial----------------------------------------------------------------------------------------
```

Capturas de ejecución:





Video de ejecución: https://www.youtube.com/watch?v=igP2a89JABg

En este ejercicio se desarrolló y estructuró jerárquicamente una sonda espacial tridimensional en OpenGL utilizando transformaciones con GLM. El modelo parte de un cuerpo principal central al que se le asigna la matriz auxiliar modelaux para mantener la cohesión espacial. A partir de esta base se despliegan de forma independiente cuatro paneles solares (izquierdo, derecho, frontal y trasero) con rotaciones controladas mediante entradas de teclado (getarticulacion20 a getarticulacion23) y colores característicos. Asimismo, utilizando anidación con modelaux2, se posiciona un cilindro superior que sirve de soporte para orientar una antena parabólica interactiva, concluyendo en la parte inferior con un módulo propulsor que respeta fielmente la jerarquía del conjunto espacial.

Para ambos ejercicios, el del rover y la sonda espacial, se ampliaron las clases Window.h y Window.cpp incorporando variables de control e inicialización para un total de 24 articulaciones independientes. Gracias a esta actualización, cada componente interactivo de ambos modelos tridimensionales como los múltiples grados de libertad de las extremidades y ruedas del rover, así como los paneles solares y la antena parabólica de la sonda, se pueden recibir y procesar de manera fluida incrementos de rotación en tiempo real mediante las teclas asignadas en el callback de teclado, logrando un control interactivo completo y preciso sobre la jerarquía gráfica.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

No se presentaron problemas, más allá de la limitante del tiempo.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.
b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
c. Conclusión

Tras la realización de esta práctica, considero que los ejercicios a realizar fueron de gran ayuda para comprender de mejor manera el tema de modelado jerárquico, ya que conllevaron un cierto nivel de complejidad además de ocupar mucho tiempo para su realización; sin embargo, lamentablemente no logré terminarlo en tiempo por cuestiones externas. A pesar de todo, fue una muy buena práctica.

1. Bibliografía en formato APA

Angel, E., & Shreiner, D. (2014). Interactive computer graphics: A top-down approach with shader-based OpenGL (6.ª ed.). Pearson.

Lengyel, E. (2019). Mathematics for 3D game programming and computer graphics (3.ª ed.). Cengage Learning.

Shirley, P., & Marschner, S. (2009). Fundamentals of computer graphics (3.ª ed.). A K Peters/CRC Press.

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL programming guide: The official guide to learning OpenGL, version 4.3 (8.ª ed.). Addison-Wesley.

The Khronos Group. (s.f.). OpenGL API Reference Pages. The Khronos Group. Recuperado de https://www.khronos.org/registry/OpenGL-Refpages/